

Introduction to DL and GenAI Project

Author: Aryan Chauhan Roll Number: 23f3003877

1. Executive Summary

The project focuses on **audio genre classification**, with the goal of assigning one of the following 10 possible classes to each audio segment: blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock. The model should be robust enough to handle noise, audio of different durations and cross genre mashups.

I experimented with three primary architectures and various ensembles, focusing on maximizing Macro F1 score while considering inference time.

Model	Parameters	Macro F1 score	Best For	Inference Time (3k samples)
Scratch CNN	6.3M	0.98540	Fast, light, and decent accuracy	1 min
ResNet-50	23.5M	0.98729	Fast, moderate, and great accuracy	1 min 6 sec
AST	86.2M	0.98720	Slow, heavy, and great accuracy	6 min 30 sec

Key Ensemble Results:

- **Scratch CNN + ResNet-50:** Achieved an excellent **0.99068 Macro F1** in just **2 minutes**.
- **Scratch CNN + ResNet-50 + AST (Best Result):** Achieved the highest score of **0.99599 Macro F1** but required **7 minutes 30 seconds**.

Kaggle Competition Performance:

- **Public leaderboard 0.99599, Rank 1**

Key Findings: Models like AST with huge parameters could not outperform simpler architectures like ResNet-50 and using ensembles of 2 or more models gave a significant boost.

2. Introduction

Problem statement

The Kaggle statement required building a classifier that can accurately predict the genre from song samples. Classifier should be capable of handling:

- Cross-song stem recombination.

- Tempo variations introduced during synchronization.
- Instrument balance changes.
- Additive environmental and synthetic noise.

Project Objective

1. To create training samples from given song stems and noise samples which can match the testing data accurately.
2. Try different architectures and parameters to get a competitive position on the leaderboard.
3. Write and maintain modular production ready code.
4. Use modern DL tools like Hugging face, pytorch lightning and wandb.

3. Dataset & Preprocessing

The raw training data consists of 100 songs distributed evenly across 10 genres. Each song is separated into 4 distinct audio stems:

- `drums.wav`
- `vocals.wav`
- `bass.wav`
- `others.wav`

The hidden test data presents several distinct domain shifts: songs range from 6 to 30 seconds in length, feature cross-song stem recombination, exhibit tempo variations, have altered instrument balances, and contain environmental noise (sourced from ESC-50).

To train a robust model capable of generalizing to these conditions, a highly diverse custom dataset was synthesized to closely mirror the test distribution.

Data Synthesis & Augmentation Pipeline

To generate a single training sample, the following dynamic pipeline was applied:

1. Stem Recombination: All 4 stems are chosen randomly within a target genre to create a completely new track, ensuring an equal number of training samples per genre.
2. Duration Sampling: The final audio length is dynamically selected:
 - 50% probability: 30-second sample
 - 40% probability: 24 to 30-second sample
 - 10% probability: 6 to 24-second sample
3. Instrument Balancing (40% probability): Randomly scales the amplitude of individual stems using a balance coefficient ranging from 0.4 to 1.0.
4. Beat Synchronization: Computes the BPM for all 4 stems, randomly selects one stem as the anchor, and synchronizes the remaining stems to that tempo using PyTorch interpolation (`torch.nn.functional.interpolate`).
5. Noise Injection (70% probability): A random 5-second noise clip is superimposed onto a random segment of the generated audio with random intensity.

Audio Preprocessing & Feature Extraction

- **Chunking:** The synthesized audio is sliced into uniform 10.24-second chunks (with shorter clips padded appropriately).
- **Mel-Spectrogram Generation:** These chunks are converted into log-mel spectrograms.
- **Design Rationale:** The 16,000 Hz sample rate and 10.24-second window were specifically chosen to create a highly optimized, standardized input shape across the project's models. At 16kHz, 10.24 seconds yields exactly 163,840 samples. Paired with a hop length of 160 and 128 mel-bins, this extracts a log-mel spectrogram of shape 128x1024. While multiple architectures are utilized, this specific dimension serves as the mathematically perfect input for the Audio Spectrogram Transformer (AST), making it an ideal anchor for the entire preprocessing pipeline.

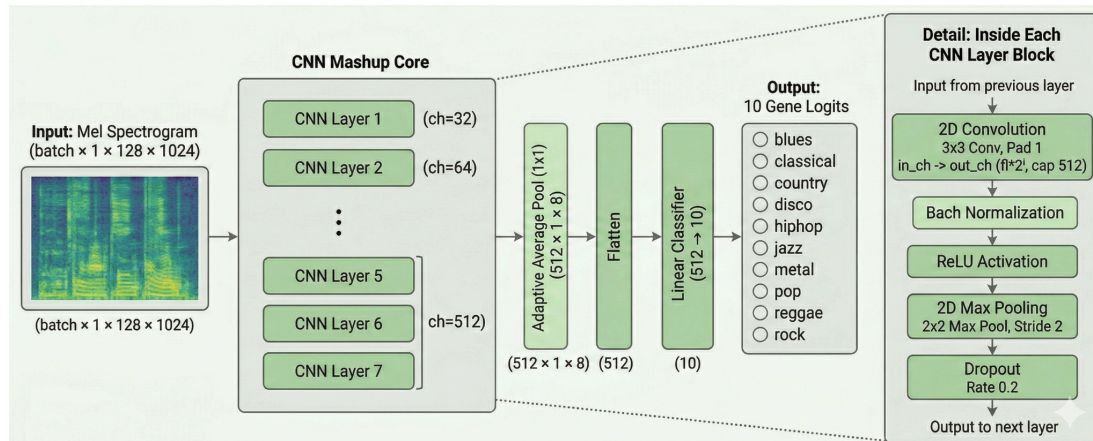
Final Dataset Statistics

- Total Generated Samples: 127,223
- Training Split: 101,771 samples
- Validation Split: 25,452 samples

4. Modeling & Experimentation

Model 1. Scratch CNN

Simple CNN with 6.3M parameters it is faster to train and inference while having performance extremely close to other models with much more parameters.



Layer details:

- 2D conv (find pattern in images)
- Batch normalization (prevent vanishing gradient and helps to converge faster)
- Relu (add non-linearity)
- 2D max pool (Downsamples spatial dimensions to reduce computational complexity)
- Dropout (Applies regularization to prevent overfitting on the training data.)

Classification head:

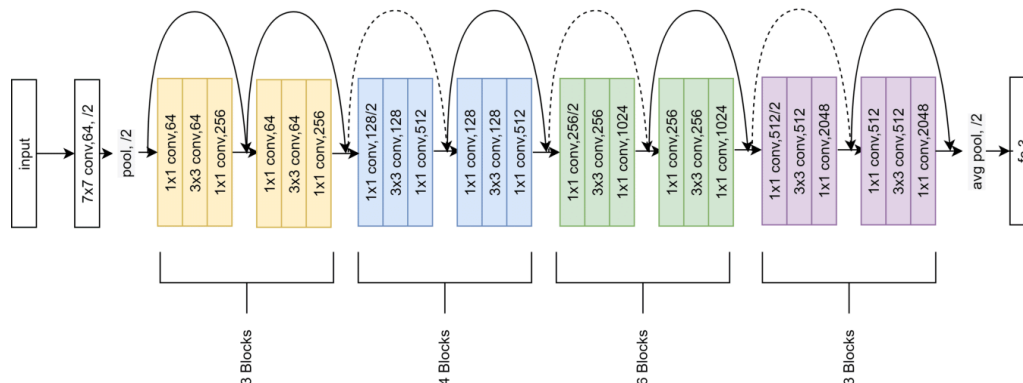
- Adaptive Average Pool (calculate avg to find 1 float for each channel)
- Flatten (Flatten all 512 floats)
- Linear Classifier (512 $\rightarrow$ 10 $\rightarrow$ genre logits)

Training Strategy:

- Optimizer AdamW (lr=0.001, weight_decay=0.01)
- CrossEntropyLoss (Balanced samples)
- Scheduler ReduceLROnPlateau (factor=0.5, patience=2)
- 40 epoch and 5 epoch patience for early stopping.
- Batch size 124
- Early stopping triggered at epoch 29

Model 2. ResNet-50

Residual network having 23.5M parameters with skipped connections to make deep architecture possible.



Architecture Details:

- The model consists of 50 layers, primarily organized into convolutional "Bottleneck" blocks with skip connections.
- Skip (or residual) connections prevent the vanishing gradient problem, which helps in effectively training much deeper models.
- Steps inside each Bottleneck block:
 - i. 1x1 Conv layer: Reduces input channels for more efficient and faster calculations.
 - ii. Batch Norm & ReLU: Normalizes data and adds non-linearity.
 - iii. 3x3 Conv layer: Extracts spatial features and finds patterns in the images.
 - iv. Batch Norm & ReLU: Normalizes and adds non-linearity.
 - v. 1x1 Conv layer: Increases input channels to match the required dimensions for the next block.
 - vi. Skip Connection: The original block input is added directly to this output before the final ReLU activation.

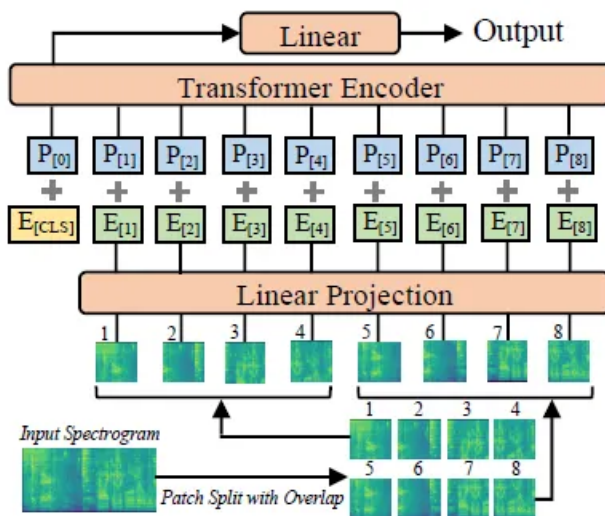
Training Strategy:

- Optimizer AdamW (lr=0.001, weight_decay=0.01)
- CrossEntropyLoss (Balanced samples)
- Scheduler ReduceLROnPlateau (factor=0.5, patience=2)

- 40 epoch and 5 epoch patience for early stopping.
- Batch size 64
- Early stopping triggered at epoch 30

Model 3. AST (Audio Spectrogram Transformer)

Audio Spectrogram Transformer with 86.2M parameters. A convolution free fully attention based model trained on audio log mel-spectrograms.



Model Architecture:

- Audio log mel-spectrograms with 128 frequency bands is used as an input.
- The spectrogram is split into 16x16 patches with an overlap of 6 in both time and frequency dimension.
- Each 16x16 patch is flattened to make an embedding of size 768.
- A learnable positional embedding of size 768 is added to each embedding
- A [CLS] token is appended at the beginning of the sequence. This [CLS] token will be used in the classification layer at the end of the transformer sequences.
- Sequence is passed to 12 transformer encoding layers.
- [CLS] token is retrieved from the sequence and then we apply a classification layer to get logits for all 10 classes.

Training Strategy:

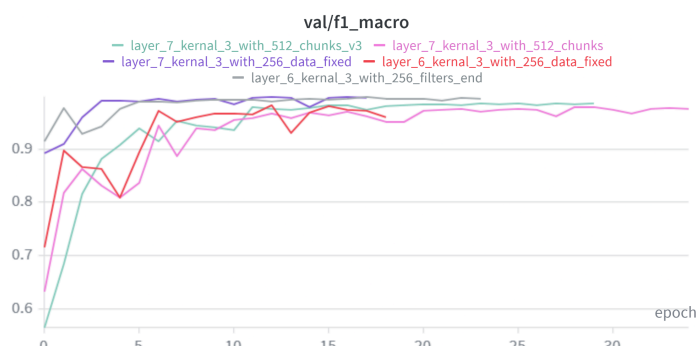
- Optimizer for backbone AdamW ($lr=1e-5$, $weight_decay=1e-4$)
- Optimizer for head AdamW ($lr=5e-5$, $weight_decay=1e-4$)
- CrossEntropyLoss (Balanced samples)
- Scheduler CosineAnnealingLR($T_{max}=8$)
- 40 epoch and 5 epoch patience for early stopping.
- Batch size 16
- Early stopping triggered at epoch 10

5. Performance & Comparative Analysis

Macro F1-score is used as the Evaluation Metric for kaggle competition as well as validation score for model tracking and early stopping.

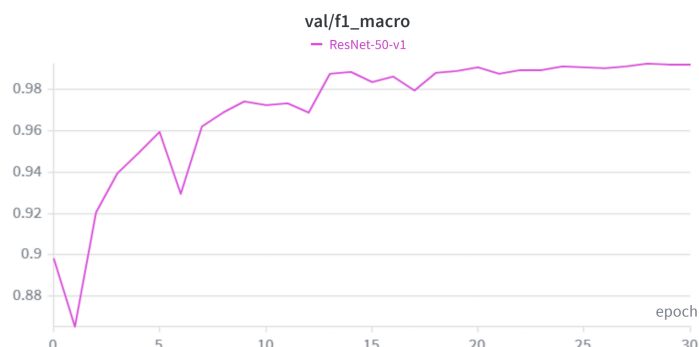
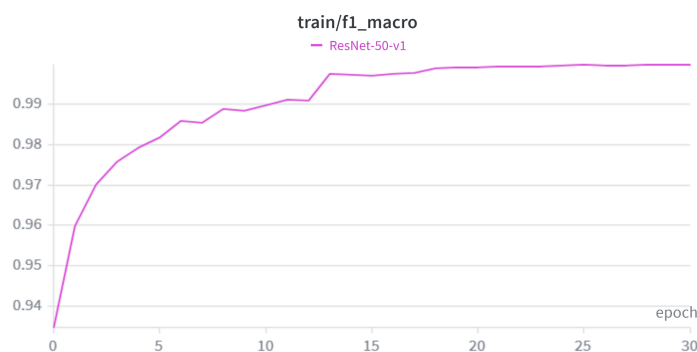
Training performance

Model 1. Scratch CNN



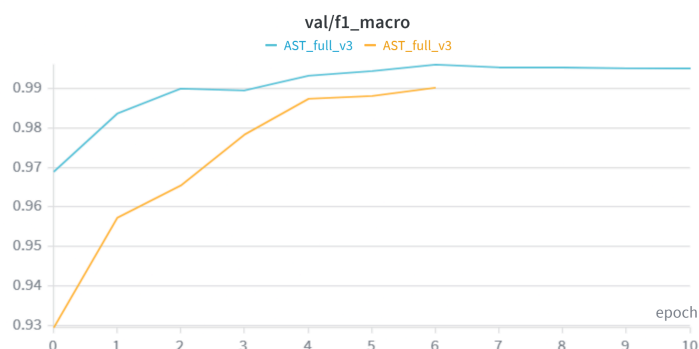
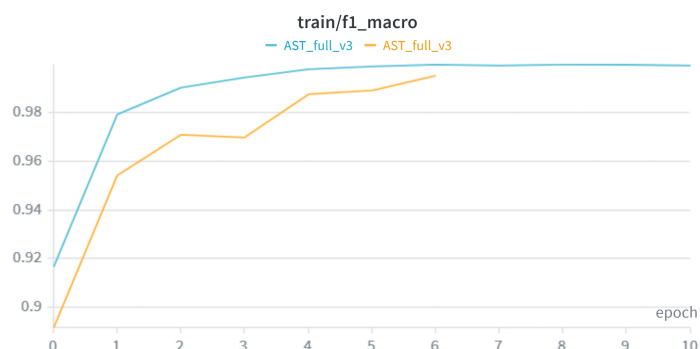
Training time: 5min per epoch (P100 GPU)

Model 2. ResNet-50



Training time: 30min per epoch (P100 GPU)

Model 3. AST (Audio Spectrogram Transformer)



Training time: 4hr per epoch (P100 GPU)

6. Conclusion & Future Work

Key Learnings

1. Generating mashups and synthetic data from audio stems.
2. Handling of audio data and using mel-spectrograms as features to train vision models.
3. Data synthesis and augmentation yields better results than trying a complex model.
4. More complex and larger models don't always out perform simpler models.

Challenges faced

1. Creating synthetic data that can represent test data closely.
2. Fine tuning larger models like AST.
3. Creating large data from given stems and managing everything while considering kaggle storage limits.
4. Limited hyperparameter search due to compute budget.

Areas for Improvement

1. Training AST or ResNet-50 with different schedulers and different combinations of lr and scheduler's parameters might improve performance slightly.
2. If allowed to use training data outside of the competition then fine tuning on modern songs.

7. Model Deployment

I have deployed an ensemble of Scratch CNN and ResNet-50 on hugging face as both of these models are fast to inference and their ensemble has a high macro f1 score.

This deployed ensemble predicts the genre of a 3min audio in less than **8sec**.

8. Key References

-
- [Audio Deep Learning Made Simple](#)
 - [Deep Residual Learning for Image Recognition](#)
 - [Audio Spectrogram Transformer](#)
 - [Hugging face AST](#)